Wordle, Romanian Edition

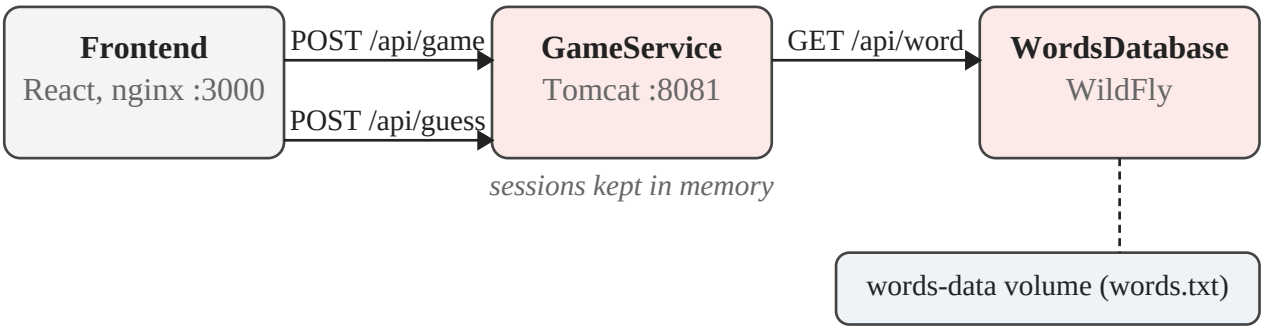
Servlet assignment · Bodo Bogdan-Mihai, Group 244

Overview

This is a Wordle clone for Romanian five-letter words, built as two servlets that talk to each other over HTTP, plus a small React interface. The player has six attempts; after each one the game colours every letter green (right place), yellow (right letter, wrong place) or grey (not in the word). Everything starts with a single `docker compose up`, and each part runs in its own container.

Architecture

The browser only ever talks to **GameService**, which runs on **Tomcat**. When a new game begins, GameService asks **WordsDatabase**, running on **WildFly**, for a random word. The secret word stays on the server and is revealed only once the game is over. WordsDatabase reads its list from a shared Docker volume that a short-lived helper container fills with `words.txt`, and it is not published outside the Docker network.



How it works

A game is a server-side session: GameService generates a UUID, stores the secret word with an attempt counter in a thread-safe in-memory map, and hands only the UUID back to the client. Each guess is validated (known session, game still running, exactly five letters), then scored by a classic two-pass comparison. The first pass marks exact matches and reserves those positions; the second pass walks the remaining letters and marks one as misplaced only if an unreserved copy of it still exists in the secret word. This is what keeps duplicate letters honest: guessing a word with two A's against a secret with one A lights up only one of them.

Alternatives and limits. A single WAR with every servlet would have been simpler, but it would not exercise two application servers. Message queues between the services (for example RabbitMQ) would decouple them better and are a natural next step, as would a shared session store such as Redis to survive restarts and allow several GameService instances. As it stands, any five letters are accepted as a guess (there is no dictionary check) and sessions are lost when GameService restarts.

Components

WordsDatabase (WildFly)

`WordRepository`	Singleton. `getInstance(): WordRepository`, `randomWord(): String`, `size(): int`. Loads the list once from `/data/words.txt` (shared volume) or from the classpath when run without Docker; trims, upper-cases, keeps only five-letter alphabetic entries and sorts them. Throws `IllegalStateException` if the file is missing, unreadable or yields no valid word.
`WordServlet`	`GET /api/word` takes no parameters and returns `{"word":"CARTE"}`.

GameService (Tomcat)

`GameSession`	Constructor `GameSession(String sessionId, String word)`. Methods `getAttemptsRemaining(): int`, `incrementAttempts(): void`, `getStatus(): GameStatus`, `setStatus(GameStatus status): void`. Status is `IN_PROGRESS`, `WON` or `LOST`; the limit is six attempts.
`SessionStore`	Singleton over a `ConcurrentHashMap`. `getInstance()`, `save(GameSession session): void`, `get(String sessionId): GameSession` (null when unknown).
`WordsDbClient`	`fetchWord(): String` calls WordsDatabase with a five-second timeout (URL from the `WORDS_DB_URL` variable). Throws `IOException` on network failure or a non-200 reply, and `InterruptedException` if interrupted.
`GuessEvaluator`	`evaluate(String word, String guess): LetterResult[]` returns `CORRECT`, `MISPLACED` or `WRONG` per position. Both strings must have the same length.
`GameServlet`	`POST /api/game` fetches a word, creates a session and returns its id, word length and attempt limit. Answers `503` if WordsDatabase is unreachable.
`GuessServlet`	`POST /api/guess` validates and scores a guess. Answers `400` (missing id or wrong length), `404` (unknown session) or `409` (game already over). Malformed JSON is not handled separately.
`CorsFilter`	Adds CORS headers to every response and answers preflight `OPTIONS` requests, since the frontend lives on another origin.

Frontend (React, served by nginx)

`App`	Holds the game state (session, submitted rows, current input, status, per-letter colours). Starts a game on load and listens to the physical keyboard as well as the on-screen one. A letter keeps its best colour across guesses (green over yellow over grey).
`Board / Keyboard`	Presentational components: a 6x5 grid of tiles and the QWERTY keys, both coloured from the state above.
`api.js`	`startGame(): Promise` and `submitGuess(sessionId, input): Promise`. Both throw an `Error` with the server message when the response is not OK. The base URL comes from `VITE_GAME_SERVICE_URL`.

API and usage examples

Starting a game and sending a guess (the secret here happens to be a word with an O in the fourth position):

```
POST /api/game
  ->  {"sessionId":"3f2c...","wordLength":5,"maxAttempts":6,
       "attemptsRemaining":6}

POST /api/guess {"sessionId":"3f2c...","input":"AEIOU"}
  ->  {"input":["A","E","I","O","U"],
       "response":["wrong","misplaced","wrong","correct","wrong"],
       "attemptsRemaining":5,"status":"IN_PROGRESS"}
```

The final reply of a finished game additionally carries the secret word in a `word` field.

Running it

Requirements: Docker with Compose (Java 17 and Node 20 are used inside the build images, so nothing else needs installing). From the project root run `docker compose up --build`. Compose starts the helper that fills the volume, then WordsDatabase, then GameService, then the frontend. The game is then at `http://localhost:3000` and GameService at `http://localhost:8081/gameservice`.

Tests

The core behaviour was checked by hand against the running stack:

Start a new game	A session id is returned and the board accepts input.
Guess sharing no letters with the secret	All five tiles turn grey.
Guess the secret word	All tiles turn green, status becomes `WON`, the victory banner appears.
Duplicate letters (two A's guessed, one in the secret)	Only one A is coloured; the other is grey.
Six wrong guesses	Status becomes `LOST` and the secret word is shown.
Stop WordsDatabase, then start a game	GameService answers 503; the rest of the app keeps working.